# Scientific Computing
# Part 1: Heat conduction in a rod

**Lukas Einkemmer**
Department of Mathematics
University of Innsbruck

# Goal

We mentioned earlier that scientific computing consists of

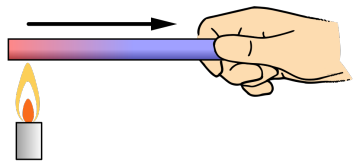
- obtaining a mathematical model for a given problem (modeling)
- discretizing the model using an appropriate numerical method
- implementing the numerical method on a computer
- solving the model on the computer
- visualization and interpretation of the results

**We will now exemplify this for the simple problem of heat conduction in a rod.**

# Obtaining a mathematical model

# Thermal conduction in a rod

We are interested in heat conduction in a rod.



Heat conduction is modeled by Fourier's law

$$q = -k \nabla T.$$

- q is the local heat flux density [$\mathrm{W\,m^{-2}}$]
- k is the material's thermal conductivity [$\mathrm{W\,m^{-1}\,K^{-1}}$]
- T is the temperature [K]

Picture by MikeRun (CC-BY-SA-4.0), https://tinyurl.com/y2tyjw2j

# Further simplifications

**Assumption:** Cross section of the rod is much smaller than its length.
**Consequence:** Temperature $T(x)$ and heat flux $q(x)$ only depends on the x-direction.

We obtain a one-dimensional problem

$$q = -k\partial_x T.$$

# Conservation laws

A fundamental idea in modeling is to consider conserved quantities.

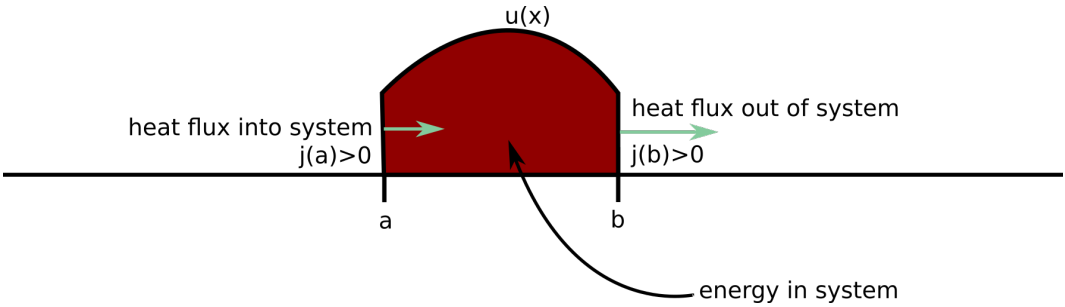
- E.g., in a closed system energy is conserved.
- Example of a global conservation law.

A local conservation law is usually associated to a global conservation law.

Energy in the system at time $t_1$ = Energy in the system at time $t_2$ + heat flux into the system - heat flux out of the system

# Conservation laws

We have a conserved quantity $u(t, x)$ and a flux $j(t, x)$ of that quantity.




Change in $u$ can be written as

$$\int_a^b u(t + \Delta t, x)\,\mathrm{d}x = \int_a^b u(t, x)\,\mathrm{d}x + \int_t^{t+\Delta t} j(s, a)\,\mathrm{d}s - \int_t^{t+\Delta t} j(s, b)\,\mathrm{d}s$$

# Conservation laws

From

$$\int_a^b u(t+\Delta t, x)\,\mathrm{d}x = \int_a^b u(t,x)\,\mathrm{d}x + \int_t^{t+\Delta t} j(s,a)\,\mathrm{d}s - \int_t^{t+\Delta t} j(s,b)\,\mathrm{d}s$$

we get

$$\int_a^b \frac{u(t+\Delta t, x) - u(t,x)}{\Delta t}\,\mathrm{d}x = -\frac{1}{\Delta t}\int_t^{t+\Delta t}\int_a^b \partial_x j(s,x)\,\mathrm{d}x\,\mathrm{d}s.$$

For $\Delta t \to 0$ we get

$$\int_a^b \left(\partial_t u(t,x) + \partial_x j(t,x)\right)\mathrm{d}x = 0.$$

# Continuity equation

We have

$$\int_a^b \left(\partial_t u(t,x) + \partial_x j(t,x)\right) \mathrm{d}x = 0.$$

Since $a$ and $b$ are arbitrary the integrand must vanish

$$\partial_t u(t,x) + \partial_x j(t,x) = 0.$$

This is called a **continuity equation**.

Writing $j$ as a function of $u$ determines the dynamics of the system.

- Depends on the specific physical problem that is considered.

# Heat equation

Internal energy $u$ is our conserved quantity.

Heat flux is given by Fourier's law

$$q(t,x) = -k\partial_x T(t,x).$$

Temperature and internal energy are related by $u = c_p \rho T$.

- $c_p$ is the specific heat capacity of the material $[\mathrm{J\,K^{-1}\,kg^{-1}}]$
- $\rho$ is the density of the material $[\mathrm{kg\,m^{-3}}]$

**Heat equation**

$$\partial_t T(t,x) = \frac{k}{\rho c_p}\partial_{xx} T(t,x).$$

# Parameters

We now have four parameters: length of the rod (L), $\rho$, $c_p$, $k$.

If we want to explore the entire parameter space and sample $n$ values for each parameter, we need $n^4$ simulations.

**This would be expensive** even for this simple problem.

- 100 values for each parameter means running $10^8$ simulation.

Fortunately not all parameters are independent.

# Nondimensionalization

The heat equation can be written as

$$\partial_t T = \alpha \partial_{xx} T, \qquad x \in [0, L]$$

with $[\alpha] = [k/(\rho c_p)] = \mathrm{m}^2/s$ the thermal diffusivity.

We now choose units such that

- $\alpha = 1$
- length of the rod is equal to 1

This gives us a reference length $L$ (the length of the rod) and a reference time

$$\tau = \frac{L^2}{\alpha}.$$

# Nondimensionalization

We call the physical units $t$ and $x$ and the nondimensional units $t'$ and $x'$.

- Note that $[t'] = 1$ and $[x'] = 1$.

Conversion between units

$$t' = \frac{t}{\tau}, \qquad x' = \frac{x}{L}.$$

Heat equation in nondimensionalized units

$$\partial_{t'} T(t', x') = \partial_{x'x'} T(t', x'), \quad x \in [0, 1].$$

All base units (length and time) relevant for the problem are uniquely determined.
**There are no parameters left!**

**A single simulation can be related to all physical configurations by simply rescaling the numerical solution** (assuming the same initial and boundary values are imposed).

# Nondimensionalization

**Step 1:** Compute a numerical solution of

$$\partial_{t'} T(t', x') = \partial_{x'x'} T(t', x').$$

Note: It is very common to write $t$ and $x$ even for nondimensionalized units.

**Step 2:** The solution in terms of physical time [s] and position [m] is given by

$$T(t/\tau, x/L).$$

# Nondimensionalization

**Example 1:** Iron rod with length $L = 0.5\,\mathrm{m}$. Then $\alpha = 2.3 \cdot 10^{-5}\,\mathrm{m}^2/\mathrm{s}$. Reference time

$$\tau = \frac{L^2}{\alpha} \approx 10^4\,\mathrm{s}.$$

One unit of nondimensional time corresponds to approximately 3 hours.

**Example 2:** Wood rod with length $L = 0.5\,\mathrm{m}$. Then $\alpha = 8 \cdot 10^{-8}\,\mathrm{m}^2/\mathrm{s}$. Reference time

$$\tau = \frac{L^2}{\alpha} \approx 3 \cdot 10^6\,\mathrm{s}.$$

One unit of nondimensional time corresponds to approximately 35 days.

**Example 3:** Wood rod with length $L = 5\,\mathrm{m}$. Reference time

$$\tau = \frac{L^2}{\alpha} \approx 3 \cdot 10^8\,\mathrm{s}.$$

Diffusion takes 100 times as long to propagate from one end of the rod to the other.

# Boundary & initial condition

To obtain a unique solution we need an initial

$$T(0, x) = T_0(x)$$

and boundary conditions.

For a fixed temperature at the left and right endpoints we have Dirichlet conditions, e.g.

$$T(t, 0) = 350, \qquad T(t, 1) = 250.$$

We could also impose a specific heat flux, which results in Neumann condition.

# Exercises

(1.1) Prove (by contradiction) that for a continuous function $f$ it holds that

$$\forall a, b \quad \int_a^b f(x)\,\mathrm{d}x = 0 \quad \Rightarrow \quad f(x) = 0.$$

(1.2) Show that $\theta(t, x) = T(t', x')$ satisfies the heat equation

$$\partial_t \theta(t, x) = \alpha \partial_{xx} \theta(t, x).$$

(1.3) Use the modeling assumption that the temperature in the rod is independent of $y$ and $z$ to compute $q_2$ and $q_3$. Provide a physical interpretation of the result.

# Exercises

(1.4) A common model for heat transfer from a solid to a surrounding fluid is

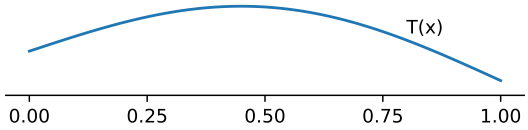
$$q = h(T_2 - T_1),$$

where $T_2$ is the temperature of the surrounding fluid and $T_1$ is the temperature of the solid surface. The constant $h$ is called the heat transfer coefficient. Determine the boundary conditions for our rod under the assumption that the left endpoint is kept at 100 C while the right endpoint is surrounded by air. For the latter we assume $T_2 = 20$. Discuss under what conditions assuming that $T_2$ is constant is reasonable.

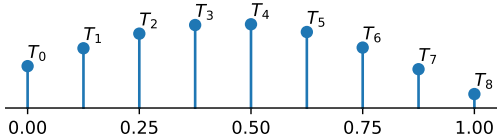
# Discretization of the model

# The grid

We start with the computational domain $[0, 1]$




We only consider function values at the grid points $x_i = i\Delta x$, where $i = 0 \ldots N$ and $\Delta x = 1/N$ is called the grid spacing.




The **numerical solution** is represented by $T_i \approx T(x_i)$.

We say that the discretization has $N + 1$ degrees of freedom.

# Spatial derivative

We start with the heat equation

$$\partial_t T(t,x) = \partial_{xx} T(t,x)$$

and restrict it to our grid

$$\partial_t T(t,x_i) = \partial_{xx} T(t,x_i), \qquad \text{for all } i = 0 \dots N.$$

We have to compute $\partial_{xx} T(t,x_i)$ using only the values at the grid points $T(t,x_i)$.

We use **numerical differentiation** with **centered differences**

$$\partial_{xx} T(t,x_i) = \frac{T(t,x_i+\Delta x) - 2T(t,x_i) + T(t,x_i-\Delta x)}{\Delta x^2} + \mathcal{O}(\Delta x^2).$$

or

$$\partial_{xx} T(t,x_i) = \frac{T(t,x_{i+1}) - 2T(t,x_i) + T(t,x_{i-1})}{\Delta x^2} + \mathcal{O}(\Delta x^2).$$

# Derivation of the centered differences

Ansatz

$$\partial_{xx} T(x_i) = \gamma_1 T(x_i + \Delta x) + \gamma_0 T(x_i) + \gamma_{-1} T(x_i - \Delta x)$$

By performing a Taylor expansion in $\Delta x$ we obtain

$$\begin{aligned}
&\gamma_1 T(x_i + \Delta x) + \gamma_0 T(x_i) + \gamma_{-1} T(x_i - \Delta x) \\
&= (\gamma_1 + \gamma_0 + \gamma_{-1}) T(x_i) + \Delta x (\gamma_1 - \gamma_{-1}) T'(x_i) \\
&\quad + \frac{\Delta x^2}{2} (\gamma_1 + \gamma_{-1}) T''(x_i) + \frac{\Delta x^3}{6} (\gamma_1 - \gamma_{-1}) T^{(3)}(x_i) + \mathcal{O}(\Delta x^4)
\end{aligned}$$

Since we want to approximate the second derivative we impose

$$\begin{aligned}
\gamma_1 + \gamma_0 + \gamma_{-1} &= 0 \\
\gamma_1 - \gamma_{-1} &= 0 \\
\gamma_1 + \gamma_{-1} &= 1
\end{aligned}$$

which yields

$$\gamma_1 = \tfrac{1}{2}, \qquad \gamma_0 = -1, \qquad \gamma_{-1} = \tfrac{1}{2}.$$

# Derivation of the centered differences

We have

$$\frac{1}{2}T(x_i + \Delta x) - T(x_i) + \frac{1}{2}T(x_i - \Delta x) = \frac{\Delta x^2}{2}T''(x_i) + \mathcal{O}(\Delta x^4)$$

and thus

$$T''(x_i) = \frac{T(x_i + \Delta x) - 2T(x_i) + T(x_i - \Delta x)}{\Delta x^2} + \mathcal{O}(\Delta x^2).$$

Some comments

- The grid spacing $\Delta x$ is a small parameter. The finer grid (smaller $\Delta x$) the more accurate the approximation.
- If $\Delta x$ is halved the error decreases by a factor of 4 (second order accurate).
- The order is larger than what we would naively expect. The $\Delta x^3$ term vanishes.
- To obtain higher order approximations more grid points can be used, e.g. $T(x_i - 2\Delta x)$, $T(x_i - \Delta x)$, $T(x_i)$, $T(x_i + \Delta x)$, $T(x_i + 2\Delta x)$.

# Time derivative

Our goal is to approximate $\partial_t T(t, x_i)$ to relate $T(t_{n+1}, x_i)$ to $T(t_n, x_i)$.

- Compute $T(t_{n+1}, x_i)$ from the known value $T(t_n, x_i)$.

**Numerical differentiation** with **forward differences**

$$\partial_t T(t_n, x_i) = \frac{T(t_n + \Delta t, x_i) - T(t_n, x_i)}{\Delta t} + \mathcal{O}(\Delta t),$$

where $\Delta t = t_{n+1} - t_n$ is the time step size.

# Finite difference scheme

Replacing time/space derivatives by forward/centered differences

$$\frac{T(t_n+\Delta t, x_i)-T(t_n,x_i)}{\Delta t}+\mathcal{O}(\Delta t)=\frac{T(t_n,x_{i+1})-2T(t_n,x_i)+T(t_n,x_{i-1})}{\Delta x^2}+\mathcal{O}(\Delta x^2).$$

Neglecting the small terms $\mathcal{O}(\Delta t)$ and $\mathcal{O}(\Delta x^2)$ and using the numerical approximation $T_i^n \approx T(t_n, x_i)$

$$\frac{T_i^{n+1}-T_i^n}{\Delta t}=\frac{T_{i+1}^n-2T_i^n+T_{i-1}^n}{\Delta x^2}.$$

Solving for $T_i^{n+1}$ we obtain the **numerical method**

$$T_i^{n+1}=T_i^n+\frac{\Delta t}{\Delta x^2}\left(T_{i+1}^n-2T_i^n+T_{i-1}^n\right).$$

**Explicit Euler in time (first order)** and **centered finite differences (second order)** in space.

# Time integration

We set $T_i^0 = T(0, x_i)$ and compute $T_i^1$, $T_i^2$, ... until we reach a final time $T$.

A numerical method is **accurate of order p in time and order q** in space if for all final time $T$ and all $n$ such that $T = n\Delta t$ we have

$$\text{global error} = T_i^n - T(T, x_i) = \mathcal{O}(\Delta t^p + \Delta x^q).$$

Be careful not to overshoot!

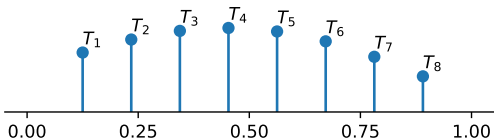
```
n = math.ceil(T/dt)
t = 0
for k in range(n):
    if T-t < dt:
        dt = T-t
    # apply the numerical method
    t += dt
```

# Dirichlet boundary conditions

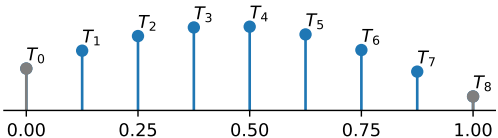
To impose the Dirichlet boundary conditions $T(t,0) = \theta_0$ and $T(t,1) = \theta_1$ we simply set $T_0^n = \theta_0$ and $T_N^n = \theta_1$ in the numerical scheme.

There are two possibilities to implement boundary conditions.

**Store interior points:** We only store the $N-1$ interior points and replace $T_0^n$ and $T_N^n$ in the stencil by $\theta_0$ and $\theta_1$.




**Store all points:** We store all $N+1$ points and set $T_0^n = \theta_0$ and $T_N^n = \theta_1$. No modification of the stencil is required.




The first approach uses slightly less memory while the second is slightly easier to implement.

# Neumann boundary conditions

Neumann boundary conditions impose the value of the derivative normal to the boundary

$$-\partial_x T(t,0) = \eta_0 \qquad \text{and} \qquad \partial_x T(t,1) = \eta_1.$$

We discretize the derivative as follows

$$\frac{T_0^n - T_1^n}{\Delta x} = \eta_0 \qquad \text{and} \qquad \frac{T_N^n - T_{N-1}^n}{\Delta x} = \eta_1$$

which we can write as

$$T_0^n = T_1^n + \eta_0 \Delta x \qquad \text{and} \, T_N^n = T_{N-1}^n + \eta_1 \Delta x.$$

Now we can use the values of $T_0^n$ and $T_N^n$ and implement either one of the schemes to impose Dirichlet boundary conditions.

# Computational effort

We have to store $T_i^{n+1}$ and $T_i^n$ in memory.

- Associated storage cost is $\mathcal{O}(N)$, where $N$ is the number of grid points.
- Choice of $\Delta x$ (and thus $N$) depends on the level of accuracy desired.

In each time step we have to process all $N$ grid points.

- Associated computational cost is $\mathcal{O}(nN)$, where $n$ is the number of time steps.

But how many time steps should we take?

- Numerical simulation indicate that numerical solution *blows up* unless we choose a very small $\Delta t$ (exercise).

# Von Neumann stability analysis

For simplicity we consider periodic boundary conditions, i.e. $T_N^n = T_0^n$, on $[0, 2\pi)$.
We write the numerical solution $T_i^n$ as a Fourier series

$$T_i^n = \sum_k \hat{T}_k^n \mathrm{e}^{\mathrm{i}kx_i}.$$

From our numerical method

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{\Delta x^2}\left(T_{i+1}^n - 2T_i^n + T_{i-1}^n\right)$$

we obtain

$$\sum_k \left(\hat{T}_k^{n+1} - \hat{T}_k^n - \frac{\Delta t}{\Delta x^2}(\mathrm{e}^{\mathrm{i}k\Delta x} - 2 + \mathrm{e}^{-\mathrm{i}k\Delta x})\hat{T}_k^n\right)\mathrm{e}^{\mathrm{i}kx_i} = 0$$

since

$$\mathrm{e}^{\mathrm{i}kx_{i+1}} = \mathrm{e}^{\mathrm{i}kx_i}\mathrm{e}^{\mathrm{i}k\Delta x} \qquad \text{and} \qquad \mathrm{e}^{\mathrm{i}kx_{i-1}} = \mathrm{e}^{\mathrm{i}kx_i}\mathrm{e}^{-\mathrm{i}k\Delta x}.$$

# Von Neumann stability analysis

Since the $e^{ikx_i}$ form a basis we have

$$\hat{T}_k^{n+1} - \hat{T}_k^n - \frac{\Delta t}{\Delta x^2}(e^{ik\Delta x} - 2 + e^{-ik\Delta x}) = 0.$$

We will now consider the so-called amplification factor

$$G_k = \frac{\hat{T}_k^{n+1}}{\hat{T}_k^n}$$

which is given by

$$\begin{aligned} G_k &= 1 + \frac{\Delta t}{\Delta x^2}(e^{ik\Delta x} - 2 + e^{-ik\Delta x}) \\ &= 1 + \frac{2\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1) \end{aligned}$$

# Von Neumann stability analysis

Since

$$|\hat{T}_k^n| = |G_k||\hat{T}_k^{n-1}| = \cdots = |G_k|^n|\hat{T}_k^0|$$

we **need for a stable numerical solution that** $|G_k| \leq 1$.

We have

$$|G_k| = \left|1 + \frac{2\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1)\right| \leq 1 \quad \Longleftrightarrow \quad \frac{2\Delta t}{\Delta x^2}(\cos(k\Delta x) - 1) \geq -2$$

which implies $\Delta t \leq \frac{1}{2}\Delta x^2$.

The **maximal allowed time step is restricted by a CFL condition**

$$\boldsymbol{\Delta t \leq \frac{1}{2}\Delta x^2}$$

which is completely independent of considerations pertaining to accuracy.

---

The CFL condition is named after Richard Courant, Kurt Friedrichs, and Hans Lewy.

# Computational effort

For fine space discretization the CFL condition forces us to take very small time steps.

Number of time steps is given by

$$n = \frac{T}{\Delta t} \geq \frac{2T}{\Delta x^2} = 2TN^2.$$

**Computational effort scales as $\mathcal{O}(N^3)$** which makes simulation very expensive.

# Exercises

(1.5) Implement the explicit Euler centered difference scheme for the heat equation with initial value $T(0,x) = 350 - 100x + 200sin(\pi x)$ and Dirichlet boundary condition $T(t,0) = 350$ and $T(t,1) = 250$. Plot the result and investigate what happens when you violate the CFL condition.

# Visualization and interpretation of the results

# Simulation

From the simulation we get a bunch of numbers

```
$ head -n 10 out-t1.data
0 350
0.00334448 349.666
0.00668896 349.331
0.0100334 348.997
0.0133779 348.662
0.0167224 348.328
0.0200669 347.994
0.0234114 347.659
0.0267559 347.325
0.0301003 346.991
```

**The purpose of computing is insight, not numbers** (Richard Hamming).

- Most of the time ...

# Plots

In this course we will use pylab/matplotlib

```
$ sudo apt install python3-matplotlib
```

Loading the data

```
from pylab import *

T = loadtxt('out-t1.data')
print(T.shape)
```

gives us an array of size $(N + 1) \times 2$

```
$ python3 plot.py
(300, 2)
```

# Plots

Basic Python code to load file and plot the data.

```
from pylab import *

T = loadtxt('out-t0.data')

figure(figsize=(8,3))
plot(T[:,0], T[:,1])
xlim([0,1])
xlabel("x'")
ylabel("temperature [K]")

# save plot into a file
savefig('T0.pdf',bbox_inches='tight', pad_inches=0.05)

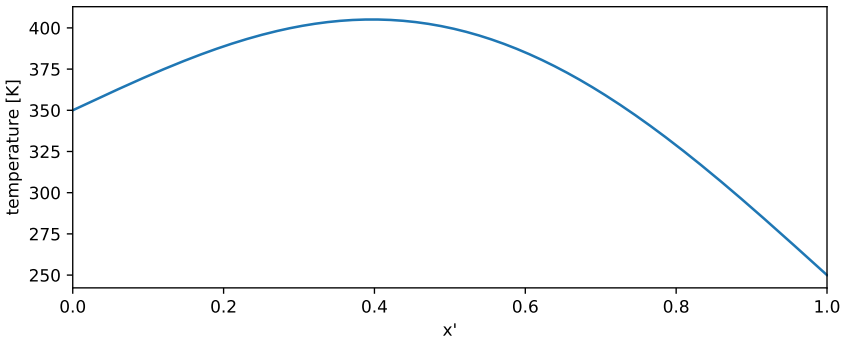
# directly show the plot
# show()
```

# Setup

**Problem setup:** Boundary conditions $T(t,0) = 350$ and $T(t,1) = 250$ and initial value

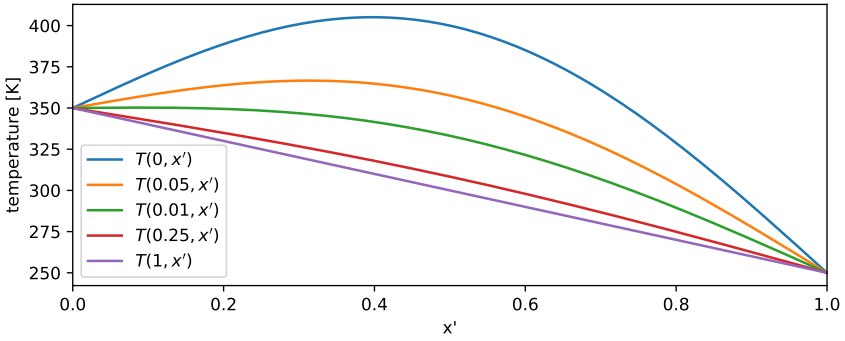
$$u(0,x') = 350 - 100x' + 100\sin(\pi x').$$

Plot at time $t = 0$:



# Solution

Solution at different points in time:



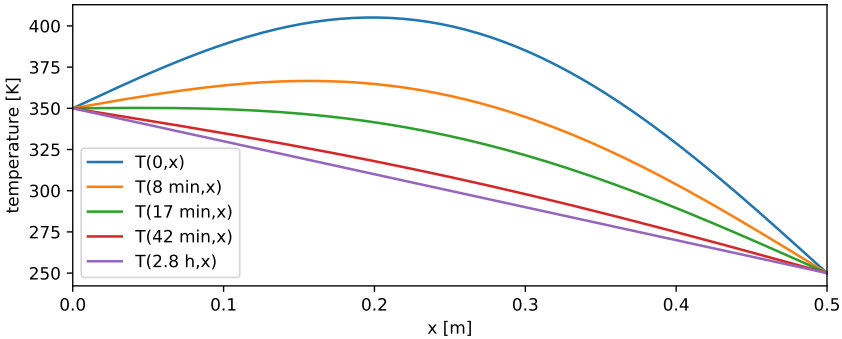

Observation:

- Variation in temperature is reduced progressively.
- Equilibrium distribution for large $t$.
- Speed of temperature change decreases as we get closer to the equilibrium.

# Iron rod

We convert back from dimensionless units $x'$ and $t'$ to physical units $x$ and $t$.

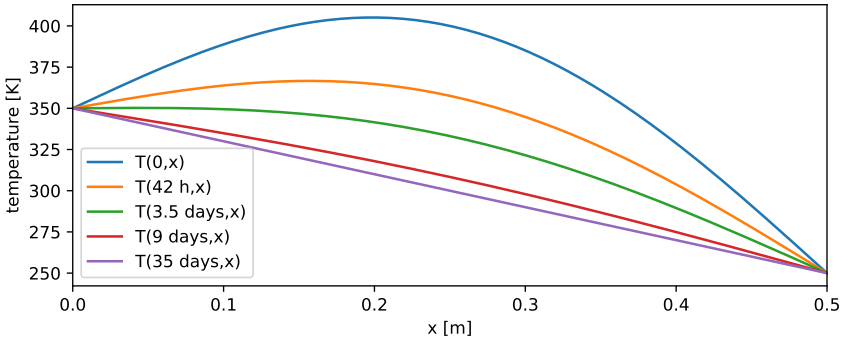
Iron rod with length 0.5 m:



# Wood rod

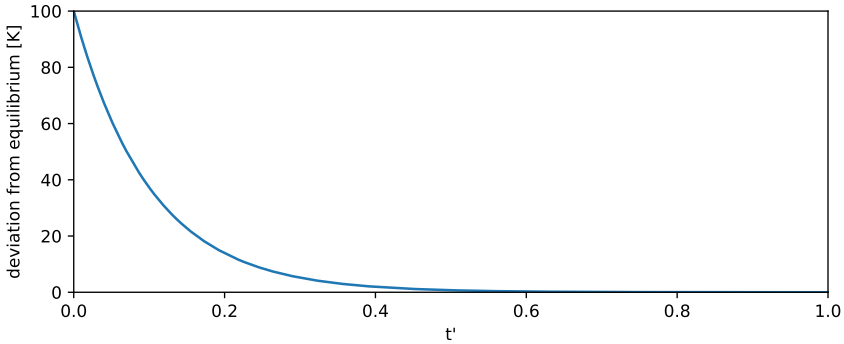
We convert back from dimensionless units $x'$ and $t'$ to physical units $x$ and $t$.

Wood rod with length 0.5 m:



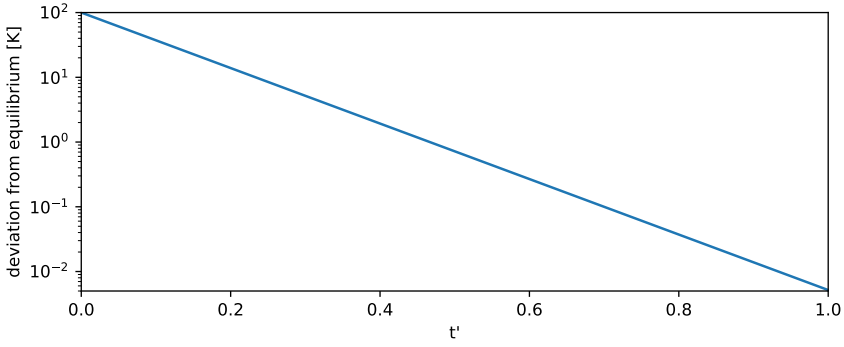
# Equilibrium

We conjecture that the equilibrium is a line connecting the two boundary values.



# Equilibrium

Better use a logarithmic $y$-axis.




Observation

- Exponential decay to equilibrium.
- Decay rate $\gamma \approx 10$, i.e. deviation $\approx 100 \exp(-\gamma t')$.

# Exercises

(1.6) Try out different initial values. How does that influence the decay rate?

(1.7) Implement the same problem but with homogeneous Neumann boundary conditions on the right end of the rod. Interpret the result.

**Play with your code!**

- One of the big advantage of scientific computing is the ability to relatively quickly experiment and test hypothesis with little repercussions.

# Algorithmic efficiency

# Computational effort for explicit Euler

We have to store $T_i^{n+1}$ and $T_i^n$ in memory.

- Associated storage cost is $\mathcal{O}(N)$, where $N$ is the number of grid points.
- Choice of $\Delta x$ (and thus $N$) depends on the level of accuracy desired.

In each time step we have to process all $N$ grid points.

- Associated computational cost is $\mathcal{O}(nN)$, where $n$ is the number of time steps.

But how many time steps should we take?

- Numerical simulation indicate that numerical solution *blows up* unless we choose a very small $\Delta t$ (exercise).

# Implicit Euler scheme

Instead of the explicit Euler scheme

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{\Delta x^2} \left( T_{i+1}^n - 2T_i^n + T_{i-1}^n \right).$$

we can use the implicit Euler scheme

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{\Delta x^2} \left( T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1} \right).$$

Notes

- Good: The implicit Euler scheme has no step size restriction (exercise).
- Bad: The implicit Euler scheme requires the solution of a linear system.
- The scheme is order 1 in time and order 2 in space.

# Computational effort

Using a LU decomposition to solve the linear system is $\mathcal{O}(N^3)$.

Number of time steps is independent of $N \Rightarrow$ **computational effort is $\mathcal{O}(N^3)$**.

- The same as for the explicit Euler method.

**Implicit methods are only advantageous if the linear system can be solved efficiently.**

- Often this has to be done with problem specific algorithms.

For the one-dimensional heat equation we can exploit the tridiagonal structure to solve the system with a computational effort of $\mathcal{O}(N)$.

# Linear algebra formulation

By putting all the degrees of freedom stored in memory into a vector $T^{n+1} = [T_1^{n+1}, \ldots, T_{N-1}^{n+1}]$ we can formulate this implicit Euler scheme as a linear system of equations.

$$\left(I - \frac{\Delta t}{\Delta x^2} A\right) T^{n+1} = T^n + F$$

with

$$A = \begin{bmatrix} -2 & 1 & 0 & \ldots & 0 \\ 1 & -2 & 1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & 1 & -2 & 1 \\ 0 & \ldots & 0 & 1 & -2 \end{bmatrix}, \qquad F = \frac{\Delta t}{\Delta x^2} \begin{bmatrix} \theta_0 \\ 0 \\ \vdots \\ \vdots \\ 0 \\ \theta_1 \end{bmatrix}.$$

This setup uses Dirichlet boundary condition with $T(t, 0) = \theta_0$ and $T(t, 1) = \theta_1$.

# Tridiagonal linear systems

**Remember:** Implicit methods are only advantageous if the linear system can be solved efficiently.

- Often this has to be done with problem specific algorithms.

We solve a linear system $Au = f$ with $A$ tridiagonal and symmetric

$$A = \begin{bmatrix} a_1 & b_1 & & & \\ b_1 & a_2 & b_2 & & \\ & b_2 & a_3 & \ddots & \\ & & \ddots & \ddots & b_{N-1} \\ & & & b_{N-1} & a_N \end{bmatrix}.$$

In our case

$$a_i = 1 + 2\frac{\Delta t}{\Delta x^2}, \qquad b_i = -\frac{\Delta t}{\Delta x^2}.$$

# Gaussian elimination

$$\begin{bmatrix} a_1 & b_1 & & & \\ b_1 & a_2 & b_2 & & \\ & b_2 & a_3 & \ddots & \\ & & \ddots & \ddots & b_{N-1} \\ & & & b_{N-1} & a_N \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ \vdots \\ f_N \end{bmatrix}$$

Add $-b_1/a_1$ times the first row to the second

$$\begin{bmatrix} a_1 & b_1 & & & \\ 0 & c_2 & b_2 & & \\ & b_2 & a_3 & \ddots & \\ & & \ddots & \ddots & b_{N-1} \\ & & & b_{N-1} & a_N \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} f_1 \\ g_2 \\ f_3 \\ \vdots \\ \vdots \\ f_N \end{bmatrix}$$

# Gaussian elimination

Add $-b_2/c_2$ times the second row to the third

$$\begin{bmatrix} a_1 & b_1 & & & \\ 0 & c_2 & b_2 & & \\ & 0 & c_3 & \ddots & \\ & & \ddots & \ddots & b_{N-1} \\ & & & b_{N-1} & a_N \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} f_1 \\ g_2 \\ g_3 \\ \vdots \\ \vdots \\ f_N \end{bmatrix}$$

Continuing the **forward sweep** we obtain

$$c_i = a_i - \frac{b_{i-1}^2}{c_{i-1}}, \qquad c_1 = a_1, \qquad i \geq 2$$

$$g_i = f_i - \frac{b_{i-1}}{c_{i-1}} g_{i-1}, \qquad g_1 = f_1, \qquad i \geq 2.$$

# Gaussian elimination

Results in an upper diagonal system

$$\begin{bmatrix} c_1 & b_1 & & & \\ 0 & c_2 & b_2 & & \\ & 0 & c_3 & \ddots & \\ & & \ddots & \ddots & b_{N-1} \\ & & & 0 & c_N \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ \vdots \\ u_N \end{bmatrix} = \begin{bmatrix} f_1 \\ g_2 \\ g_3 \\ \vdots \\ \vdots \\ g_N \end{bmatrix}$$

that can be solved by backward substitution **(backward sweep)**

$$u_i = \frac{g_i - b_i u_{i+1}}{c_i}, \qquad u_N = \frac{g_N}{c_N}, \qquad i \leq N-1.$$

The **Thomas algorithm** requires $\mathcal{O}(N)$ operation.

- We have exploited the structure of the matrix (tridiagonal) to solve the system much faster than performing a full LU decomposition.
- Example of a fast Poisson solver.

# Exercises

(1.8) Show that the implicit Euler centered difference scheme is stable for any combination of $\Delta t$ and $\Delta x$ (by performing a von Neumann stability analysis).

(1.9) Implement the implicit Euler scheme and demonstrate that it is able to take much larger time steps $\Delta t$ compared to the explicit Euler scheme. You can use scipy.linalg.solve to solve the resulting linear system.

(1.10) Implement the implicit Euler scheme using your own implementation of the Thomas algorithm.

(1.11) Implement the implicit Euler centered difference scheme for the configuration of exercise (1.5). Use scipy.linalg.solve and scipy.linalg.solve_banded to solve the linear system. Compare the run time for these two implementations with the explicit Euler scheme.

Exercise (1.11) is an advanced exercise.

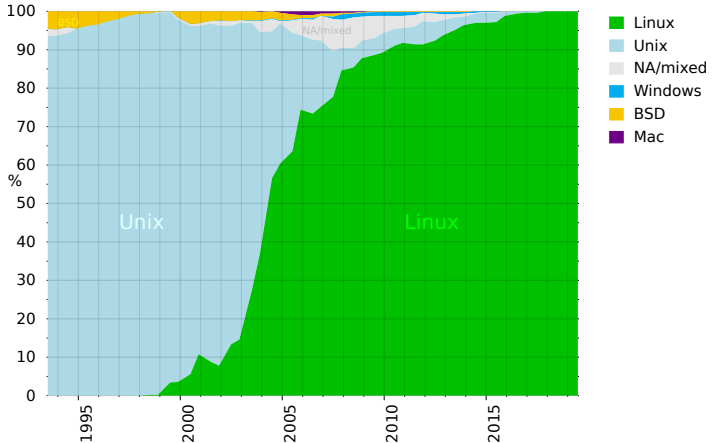
# Implementation efficiency

# Goal

What is the difference between what we have done so far and a production level code?

# Linux

**Almost all dedicated computing systems use Linux** (the picture shows market share of operating systems in the TOP500).




Access to these systems is usually provided via a command line interface (ssh).

Picture by Benedikt Seidl `https://tinyurl.com/razmkdg`

# Simulation vs. visualization

**Computation and visualization are separated.**

- The simulation code writes output to a file.
- Output files are used to generate plots (in Python or some dedicated plotting program).

# Simple stencil code

Python

```
for i in arange(1,N-1):
  for j in arange(1,N-1):
      out[j+i*N] = inn[j+i*N] \
                     + alpha*(inn[j-1+i*N] - 2.0*inn[j+i*N] + inn[j+1+i*N])
```

```
Wall time (2nd order):  9.5 s
```

C++

```
for(int i=1;i<N-1;i++)
    for(int j=1;j<N-1;j++)
        out[j+i*N] = in[j+i*N]
                       + alpha*(in[j-1+i*N] - 2.0*in[j+i*N] + in[j+1+i*N]);
```

```
Wall time (2nd order): 0.0028 s
```

# Performance

**Drastic performance improvement ($\approx$ 3000x).**

- Reduces run time from say 1h to 1s.

**Python** is an **interpreted language.**

- A program, the interpreter, reads the python code and executes one statement after another.

**C++** is a **compiled language** that uses a two-step approach

- **Compilation:** A program, the compiler, reads the C++ code and translates it into machine code.
- **Execution:** The machine code is directly executed by the CPU.

In reality Python uses an intermediate representation (bytecode).

# Assembly code

Example of a vastly **simplified assembly/machine** code

```
mov 0x38AF2 r1
mov 0xA03DD r2
iadd r1 r2
mov r2 0x38AF2
```

Arithmetic operation (iadd in this case) operate on CPU registers (r1, r2).

Memory operation (mov) read and write data from/to an address (0x38AF2, 0xA03DD) in main memory.

Modern CPUs are much more complicated.

# Compilation

Can we compile Python?

```
x = 3 # x is a integer
if condition:
    x = 3.0 # x is a floating point number
y = 2*x
```

What operation should 2*x be compiled to?

- integer multiplication
- floating point multiplication

Compiler would need to understand all possible types that x can have.

- For an interpreted language this is much simpler (can be decided during run time)

**In C++ type information need to be specified at compile time.**

- Having type information available makes optimizing the machine code easier.

# Optimizing Python

Our goal is to express the stencil by coarse grained built-in functions.

```
# roll is a Numpy function that takes an array and shifts
# it to the left or to the right.
out = inn + alpha*(roll(inn,-1) - 2.0*inn + roll(inn,1))
```

```
Wall time (2nd order, roll):  0.031 s
```

Fraction of the time spend interpreting the code is small.

- The function roll can be implemented in a compiled language.

Much faster than the Python for loop, but still significantly slower than C++ ($\approx$ 11x).

# Performance

### Fourth order stencil in Python

```
for i in arange(1,N-1):
    for j in arange(1,N-1):
        out[j+i*N] = inn[j+i*N] + alpha*(-1./12.*inn[j-2+i*N] \
          + 4./3.*inn[j-1+i*N] - 5./2.*inn[j+i*N] + 4./3.*inn[j+1+i*N] \
          - 1./12.*inn[j+2+i*N]);
```

```
Wall time (2nd order):  9.5 s
Wall time (4th order):  15.4 s
```

### Fourth order stencil in C++

```
for(int i=1;i<N-1;i++)
    for(int j=2;j<N-2;j++)
        out[j+i*N] = inn[j+i*N] + alpha*(-1./12.*in[j-2+i*N]
          + 4./3.*in[j-1+i*N] - 5./2.*in[j+i*N] + 4./3.*in[j+1+i*N]
          - 1./12.*in[j+2+i*N]);
```

```
Wall time (2nd order): 0.0028 s
Wall time (4th order): 0.0028 s
```

# Performance

In Python using the second order method is 1.6x faster than fourth order method.

In C++ the second and fourth order methods take the same time.

- Performance is limited by fetching data from memory and not by the number of arithmetic operations the CPU can perform.

**Writing a Python code to predict the performance of a C++ code is a bad idea!**

# The role of Python

Many software packages have embraced a high-level Python interface.

Python used to specify the problem but all the heavy lifting is done under the hood (usually in C++).

Makes the software approachable for users while maintaining good performance.

# Extension to two dimension

# Heat equation

So far we have only considered a one-dimensional example.

The two- and three-dimensional case is much more interesting.

The heat equation can be easily generalized to two-dimensions

$$\partial_t T(t,x,y) = \partial_{xx} T(t,x,y) + \partial_{yy} T(t,x,y) =: \Delta T(t,x,y),$$

where $\Delta$ is the Laplacian.

# Discretization

For $(x, y) \in [0, 1]^2$ we define $x_i$ and $y_j$ as before. Then

$$T_{ij}^n \approx T(t_n, x_i, y_j).$$

**Explicit Euler scheme**

$$T_{ij}^{n+1} = T_{ij}^n + \frac{\Delta t}{\Delta x^2}(T_{i+1,j}^n - 2T_{ij}^n + T_{i-1,j}^n) + \frac{\Delta t}{\Delta y^2}(T_{i,j+1}^n - 2T_{ij}^n + T_{i,j-1}^n),$$

where $\Delta x$ and $\Delta y$ is the grid spacing in the x- and y-direction, respectively.

**Implicit Euler scheme**

$$T_{ij}^{n+1} = T_{ij}^n + \frac{\Delta t}{\Delta x^2}(T_{i+1,j}^{n+1} - 2T_{ij}^{n+1} + T_{i-1,j}^{n+1}) + \frac{\Delta t}{\Delta y^2}(T_{i,j+1}^{n+1} - 2T_{ij}^{n+1} + T_{i,j-1}^{n+1}).$$

# Linear algebra formulation

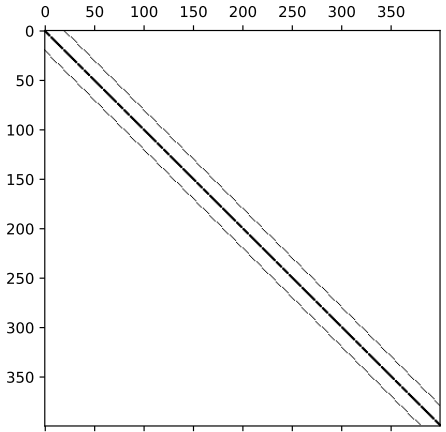
We linearize the numerical solution into a vector

$$T^n_{i+N_x j} = T^n_{ij}.$$

The implicit Euler scheme

$$\left(I - \frac{\Delta t}{\Delta x^2} I \otimes A - \frac{\Delta t}{\Delta y^2} A \otimes I\right) T^{n+1} = T^n + F,$$

where $A$ is the finite difference matrix used earlier.




**Not possible to write the discretization of the two-dimensional heat equation as a banded matrix (with small bandwidth).**

# ADI (Alternating direction implicit)

**Step 1:** Solve

$$T_{ij}^{\star} = T_{ij}^{n} + \frac{\Delta t}{\Delta x^2}\left(T_{i+1,j}^{\star} - 2T_{ij}^{\star} + T_{i-1,j}^{\star}\right).$$

For each $j$ we only have to solve a one-dimensional problem, for which we know an efficient algorithm.

**Step 2:** Solve

$$T_{ij}^{n+1} = T_{ij}^{\star} + \frac{\Delta t}{\Delta y^2}\left(T_{i,j+1}^{n+1} - 2T_{ij}^{n+1} + T_{i,j-1}^{n+1}\right),$$

which again is a one-dimensional problem for each $i$.

**ADI** scheme is **unconditionally stable** and can be implemented in $\mathcal{O}(N)$ operation.

# Exercises

(1.12) Implement the explicit Euler scheme for the two-dimensional heat equation.

(1.13) Consider the two-dimensional heat equation with initial value $u_0(x, y) = x^2 + y^2$ on the domain $[0, 1] \times [0, 1]$. The boundary conditions $u(t, 0, y) = y^2$, $u(t, 1, y) = 1 + y^2$, $u(t, x, 0) = x^2$, $u(t, x, 1) = 1 + x^2$ are imposed. Plot the result at different times. How does the equilibrium look like? Experiment with different initial values.

(1.14) Implement the ADI scheme for the two-dimensional heat equation and compare its performance to the explicit Euler scheme.

Exercise (1.14) is an advanced exercise.